❶ Includes application name, trace ID, and span ID next to the log level (%5p)

Next, open a Terminal window, navigate to the Catalog Service root folder, and run `./gradlew bootBuildImage` to package the application as a container image.

The final step is configuring and enabling the OpenTelemetry Java Agent. For simplicity, we'll enable OpenTelemetry only when running applications in containers and rely on environment variables to configure it.

We need three pieces of configuration to successfully enable tracing:

- *Instruct the JVM to load the OpenTelemetry Java agent.* We can do that via the `JAVA_TOOL_OPTIONS` standard environment variable supported by OpenJDK to provide additional configuration to the JVM.
- *Use the application name to tag and categorize traces.* We'll use the `OTEL_SERVICE_ NAME` environment variable supported by the OpenTelemetry Java agent.
- *Define the URL of the distributed tracing backend.* In our case, it's Tempo on port 4317, and it can be configured via the `OTEL_EXPORTER_OTLP_ENDPOINT` environment variable supported by the OpenTelemetry Java agent. By default, traces are sent over gRPC.

Go to your Polar Deployment project (polar-deployment), and open the Docker Compose file (docker/docker-compose.yml). Then add the necessary configuration to Catalog Service to support tracing.

Listing 13.22 Defining OpenTelemetry for the Catalog Service container

```
version: "3.8"
services:
```